Agent Mesh SRE — Blueprint & Architecture v2.1

Document version: 2.1 **Application version:** Current (surajcsibm fork, May 2026) **Last updated:** May 2026

Table of Contents

---

1. Executive Summary

Agent Mesh SRE v2.1 is a Next.js 14 (App Router) application that demonstrates autonomous AI-driven Kafka SRE operations through a **client-side simulation engine**. Unlike the upstream reference implementation (which requires a running Kafka cluster, Kubernetes, and server-side SSE streams), this fork operates entirely in the browser using a deterministic simulation of Kafka metric events, agent state transitions, and the MRAL reasoning loop.

The primary architectural change from v1.x is the replacement of server-side streaming infrastructure (`mesh.ts`, Zustand stores, SSE endpoint) with a pure client-side simulation

(`client-sim.ts` + `useMeshStream.ts` + React `useReducer`). This makes the demo instantly runnable on Vercel or any Node.js host with zero Kafka infrastructure.

v2.1 adds scenario-driven topic auto-creation, paginated topic browsing, and a fork comparison document (`FORK-COMPARISON.md`) shipped with the repository.

---

2. System Overview

```
Browser (Next.js App Router)
│
├── app/
│   ├── page.tsx              → redirect to /dashboard
│   ├── login/page.tsx        → authentication page
│   ├── dashboard/page.tsx    → main dashboard (server component wrapper)
│   └── api/
│       ├── auth/[...nextauth]/route.ts   → NextAuth handler
│       ├── send-email/route.ts           → Nodemailer SMTP route
│       └── mesh/...                      → (legacy, unused in mock mode)
│
├── components/
│   ├── Dashboard.tsx         → entire UI (client component, ~950 lines)
│   ├── AgentCanvas.tsx       → React Flow canvas (dynamically imported, SSR
off)
│   ├── useMeshStream.ts      → simulation hook (useReducer + setTimeout)
│   └── client-sim.ts         → deterministic scenario simulation engine
│
└── lib/
    ├── types.ts              → shared TypeScript interfaces
    └── emailer.ts            → Nodemailer wrapper
```

The application has no persistent database. All state lives in React `useReducer` (session state) and `localStorage` (scenario history across sessions).

---

3. Repository Structure

```
agent-mesh-sre/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── globals.css
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── dashboard/
│   │   │   └── page.tsx
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts
│   │       ├── send-email/route.ts
```

```
│   │       └── mesh/
│   │           ├── stream/route.ts
│   │           ├── approve/route.ts
│   │           └── reject/route.ts
│   ├── components/
│   │   ├── Dashboard.tsx
│   │   ├── AgentCanvas.tsx
│   │   ├── useMeshStream.ts
│   │   └── client-sim.ts
│   └── lib/
│       ├── types.ts
│       └── emailer.ts
├── FORK-COMPARISON.md        ← fork vs upstream PR #1 analysis
├── .env.local.example
├── next.config.mjs
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

---

4. Technology Stack

Layer	Technology	Version
Framework	Next.js (App Router)	14.x
Language	TypeScript	5.x
Styling	Tailwind CSS	3.x
Canvas	@xyflow/react (React Flow)	12.x
Auth	NextAuth.js	4.x
Email	Nodemailer	6.x
Runtime	Node.js	18+
Deployment	Vercel	—

No external state management library (Zustand, Redux) is used. The entire simulation runs in a single `useReducer` within `useMeshStream.ts`.

---

5. Client-Side Simulation Architecture

The simulation replaces server-side Kafka and agent infrastructure entirely.

```
useMeshStream(scenarioId)
    │
    ├── dispatch(RESET)
    ├── runScenario(scenarioId, dispatch, getState)   ← from client-sim.ts
    │       │
    │       ├── emit publish events  (INTAKE agent)
```

```
    │       ├── emit consume events  (BROKER → MONITOR)
    │       ├── emit reasoning       (MONITOR REASON phase)
    │       ├── pause for approval   (dispatch AWAITING_APPROVAL)
    │       │       └── waits for approve() / reject() call
    │       ├── emit acting          (MONITOR ACT phase)
    │       ├── emit tool-call       (MCP tool invocation)
    │       ├── emit writer events   (WRITER agent)
    │       ├── emit lesson          (MONITOR LEARN phase)
    │       └── emit notification    (NOTIFY agent → email)
    │
    └── returns { state, approve, reject, triggerTopicAction,
dismissEmailSummary }
```

`client-sim.ts` exports `runScenario(id, dispatch, getState)`. Each scenario is implemented as an `async` function using `await delay(ms)` between steps to create a realistic timeline. Events are dispatched to the reducer which updates React state, driving UI updates reactively.

---

6. Data Types and Interfaces

AgentState

```ts
interface AgentState {
  id:          string;
  name:        string;
  status:      "idle" | "monitoring" | "reasoning" | "acting" |
               "awaiting-approval" | "learning" | "killed" | "replaying";
  mralPhase?:  "monitor" | "reason" | "act" | "learn";
  lastReasoning?: {
    rootCause:   string;
    confidence:  number;       // 0.0–1.0
    candidates:  string[];
  };
  lastAction?: {
    tool:    string;
    detail:  string;
    result?: string;
  };
  lastLesson?: {
    notes:   string;
    success: boolean;
  };
}
```

KafkaTopic

```ts
interface KafkaTopic {
  id:                string;
  name:              string;
  partitions:        number;
```

```
  replicationFactor: number;
  retentionHours:    number;
  lagTotal:          number;
  msgPerSec:         number;
  status:            "healthy" | "degraded" | "critical";
  consumerGroups:    string[];
  description:       string;
}
```

EmailSummaryData

```
interface EmailSummaryData {
  scenarioLabel: string;
  approved:      boolean;
  lagBefore:     number;
  lagAfter:      number;
  reasoning:     string;
  action:        string;
  lesson:        string;
  liveEvents?:   Array<{
    type:    string;
    agent:   string;
    summary: string;
    ts:      number;
  }>;
  sent:        boolean;
  emailError?: string;
}
```

---

7. Agent State Machine

Each agent transitions through a well-defined state machine during a scenario run:

```
idle ──► monitoring ──► reasoning ──► awaiting-approval ──► acting ──►
learning ──► idle
                  └──────────────────────────────────────────────────►
                  (auto-approve path when confidence ≥ threshold)

Any state ──► killed (via Kill button)
killed    ──► idle   (via Restart button)
```

The MONITOR agent is the only agent that traverses the full MRAL cycle. INTAKE stays in `monitoring` during publish events. WRITER transitions briefly to `acting` during report generation. NOTIFY transitions to `acting` when sending email.

---

8. MRAL Loop Implementation

The MRAL loop is implemented in `client-sim.ts` as a sequence of dispatched events. Each scenario function follows this pattern:

```
// 1. MONITOR phase — INTAKE publishes events
await dispatch({ type: "AGENT_UPDATE", agent: "intake", status: "monitoring"
});
await delay(800);
await dispatch({ type: "AUDIT_EVENT", record: { type: "publish", ... }});

// 2. REASON phase
await dispatch({ type: "AGENT_UPDATE", agent: "monitor", status: "reasoning",
  mralPhase: "reason", lastReasoning: { rootCause: "...", confidence: 0.87
}});
await delay(1200);

// 3. AWAITING phase (human-in-the-loop)
await dispatch({ type: "AWAITING_APPROVAL", request: { tool:
"kafka.scaleConsumers" }});
await waitForDecision(getState);

// 4. ACT phase
await dispatch({ type: "AGENT_UPDATE", agent: "monitor", status: "acting",
  mralPhase: "act", lastAction: { tool: "...", detail: "..." }});
await delay(1000);

// 5. LEARN phase
await dispatch({ type: "AGENT_UPDATE", agent: "monitor", status: "reasoning",
  mralPhase: "learn", lastLesson: { notes: "...", success: true }});
await delay(800);

// 6. EMAIL SUMMARY
await dispatch({ type: "EMAIL_SUMMARY", summary: { scenarioLabel, approved,
... }});
```

---

9. Canvas Architecture (AgentCanvas.tsx)

`AgentCanvas.tsx` is a dynamically-imported client component (SSR disabled) that renders the agent topology using React Flow v12.

Node types

Type	Component	Description
`agent`	`AgentNode`	Primary circular agent node (200 px diameter)
`broker`	`BrokerNode`	Central Kafka broker node (160 px)
`subagent`	`SubAgentNode`	Ephemeral MRAL phase circle (90 px)

Node positions (static)

```
const NODE_POS = {
  intake:       { x: 30,  y: 20  },
  writer:       { x: 390, y: 20  },
  broker:       { x: 206, y: 180 },
  monitor:      { x: 30,  y: 340 },
  notification: { x: 390, y: 340 },
};
const SUB_AGENT_POS = { x: 41, y: 545 };  // below MONITOR
```

Accent colours

```
const ACCENT = {
  intake:       "#1D9E75",   // emerald
  monitor:      "#7c3aed",   // violet
  writer:       "#0891b2",   // cyan
  notification: "#f97316",   // orange
};
```

Click propagation fix

React Flow v12 consumes pointer events on node wrappers. Interactive elements inside nodes (Kill/Restart buttons) require both `e.stopPropagation()` on the click handler and `pointerEvents: "all"` on the element style to receive clicks.

---

10. Dashboard Architecture (Dashboard.tsx)

`Dashboard.tsx` is a single large client component (`"use client"`) that composes the entire dashboard. It imports `useMeshStream` for simulation state and `AgentCanvas` dynamically.

Key state

State	Type	Description
`state`	`MeshState`	Full simulation state from `useMeshStream`
`canvasHeight`	`number`	Canvas height (380–960 px)
`topics`	`KafkaTopic[]`	All topics — initial seed + background auto-generated + scenario-upserted
`topicsVisible`	`number`	How many topics the panel currently shows (starts at 20, +20 per click)
`summaryHistory`	`EmailSummaryData[]`	Saved scenario runs (from `localStorage`)
`historyMounted`	`boolean`	Guards SSR hydration mismatch
`activeTab`	`string`	Selected right-panel tab
`extraOpen`	`boolean`	Expands extra scenario list

Scenario auto-create: SCENARIO_AUTO_TOPICS

A static map `SCENARIO_AUTO_TOPICS` defines the Kafka topics relevant to each scenario:

```
const SCENARIO_AUTO_TOPICS: Record<string, Omit<KafkaTopic, "id">[]> = {
  "lag-spike": [
    { name: "ops.kafka.metrics.v1", partitions: 6, replicationFactor: 3,
      lagTotal: 18500, msgPerSec: 420, status: "critical", ... },
    { name: "ops.requests.v1", partitions: 3, ..., status: "healthy" },
  ],
  "controller-failover": [ ... ],
  "share-group":         [ ... ],
  // ... all 10 scenarios
};
```

handleTrigger wrapper

All scenario button clicks go through `handleTrigger` instead of `trigger` directly. This wrapper upserts scenario topics before starting the simulation:

```
const handleTrigger = (scenarioId: string) => {
  const autoTopics = SCENARIO_AUTO_TOPICS[scenarioId];
  if (autoTopics?.length) {
    setTopics(prev => {
      let next = [...prev];
      for (const t of autoTopics) {
        const idx = next.findIndex(x => x.name === t.name);
        if (idx >= 0) {
          // Update metrics in-place
          next[idx] = { ...next[idx], lagTotal: t.lagTotal,
                        msgPerSec: t.msgPerSec, status: t.status };
        } else {
          // Insert at front
          next = [{ id: `t-scen-${Date.now()}-...`, ...t }, ...next];
        }
      }
      return next;
    });
    setTopicsVisible(v => Math.max(v, 20)); // ensure panel shows at least
page 1
  }
  trigger(scenarioId);
};
```

TopicsPanel pagination

`TopicsPanel` accepts `visibleCount` and `onShowMore` props. It slices the sorted topic list to `visibleCount` and renders a “More” button when there are additional topics:

```
const visible   = sorted.slice(0, visibleCount);
const remaining = sorted.length - visibleCount;
// ...
{remaining > 0 && (
  <button onClick={onShowMore}>
    + {Math.min(20, remaining)} more topics ↓
```

```
  </button>
)}
```

Dashboard passes `topicsVisible` and `() => setTopicsVisible(v => v + 20)` into the panel.

Background topic generation

An independent `useEffect` generates a new random topic every 18–40 seconds, simulating organic topic growth in a live Kafka cluster. Topics are added only if the name is not already present.

Hydration-safe localStorage pattern

```
// Init with empty array — matches SSR output
const [summaryHistory, setSummaryHistory] = useState<EmailSummaryData[]>([]);
const [historyMounted, setHistoryMounted] = useState(false);

useEffect(() => {
  try {
    const saved = localStorage.getItem("sre-scenario-history");
    if (saved) setSummaryHistory(JSON.parse(saved));
  } catch {}
  setHistoryMounted(true);
}, []);

useEffect(() => {
  if (!historyMounted) return;
  localStorage.setItem("sre-scenario-history",
JSON.stringify(summaryHistory));
}, [summaryHistory, historyMounted]);
```

---

11. Topics Management

Topic actions dispatched to the reducer:

Action type	Effect
`TOPIC_CHANGE`	Updates lag / health for a topic after MONITOR acts
`TOPIC_HEAL`	Triggers a dedicated MRAL cycle targeting one topic
`TOPIC_CREATE`	Adds a new topic to the list (duplicate name check)
`TOPIC_COPY`	Clones a topic with `-copy` suffix

Topic seeding hierarchy (highest precedence first):

1. **Scenario auto-create** — upserted by `handleTrigger` at scenario start
2. **Manual create/copy** — user-initiated from the Topics panel
3. **Background auto-generation** — random topics every 18–40 s

4. **INITIAL_TOPICS** — 8 seed topics loaded at startup (payment / invoice domain)

---

12. Authentication (NextAuth.js)

Authentication is handled by NextAuth.js v4 with two providers:

Google OAuth — Requires `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`. Users are redirected to Google's consent screen and returned to the dashboard after successful sign-in.

Credentials provider — Email/password authentication. Includes forgot-password flow that sends a reset link via SMTP.

Middleware in `src/middleware.ts` protects `/dashboard` — unauthenticated requests are redirected to `/login`.

Session strategy is `jwt`. The JWT is signed with `NEXTAUTH_SECRET`.

---

13. Email Notification Pipeline

```
NOTIFY agent fires
     │
     ├── dispatch EMAIL_SUMMARY to useMeshStream reducer
     │        └── sets state.emailSummary
     │
     ├── Dashboard.tsx detects state.emailSummary change
     │        └── captures to summaryHistory (with live events hydrated from
auditLogRef)
     │
     └── /api/send-email route (POST)
              ├── receives EmailSummaryData JSON body
              ├── renders HTML email template
              │        ├── scenario summary table
              │        ├── lag delta (before → after)
              │        ├── reasoning / action / lesson
              │        └── live events table
              └── sends via Nodemailer (SMTP)
```

---

14. Scenario Engine (client-sim.ts)

`client-sim.ts` exports one function:

```ts
export async function runScenario(
  scenarioId: string,
  dispatch: Dispatch<MeshAction>,
  getState: () => MeshState
): Promise<void>
```

Ten scenarios are implemented. Adding a new scenario requires only:

1. An entry in `PINNED_SCENARIOS` or `EXTRA_SCENARIOS` in `Dashboard.tsx`
2. A matching entry in `SCENARIO_AUTO_TOPICS` in `Dashboard.tsx`
3. A `case "scenario-id":` block in `client-sim.ts`

No server changes needed.

---

15. State Management (useMeshStream.ts)

`useMeshStream` is a custom hook that owns all simulation state via `useReducer`.

```
interface MeshState {
  agents:           AgentState[];
  auditLog:         AuditRecord[];
  toasts:           Toast[];
  topics:           TopicMetrics[];
  approvalRequest:  ApprovalRequest | null;
  approvalDecision: boolean | null;
  emailSummary:     EmailSummaryData | null;
  running:          boolean;
}
```

The reducer handles approximately 12 action types. Sub-agent visibility is derived from `AgentState.status` / `AgentState.mralPhase` in `AgentCanvas.tsx` — it is not separate state.

---

16. Scenario History Persistence

Scenario history is stored in `localStorage` under the key `"sre-scenario-history"` as a JSON array of `EmailSummaryData` objects. Maximum 30 entries (oldest dropped beyond limit). The `historyMounted` boolean prevents the save effect from running before the load effect completes.

---

17. API Routes

Route	Method	Auth	Description
`/api/auth/[...nextauth]`	GET/POST	—	NextAuth handler
`/api/send-email`	POST	Required	Sends incident email via SMTP
`/api/mesh/stream`	GET	Required	No-op in mock mode
`/api/mesh/approve`	POST	Required	No-op in mock mode
`/api/mesh/reject`	POST	Required	No-op in mock mode

---

18. Deployment Architecture

Vercel (recommended)

```
GitHub push to main
     │
     └── Vercel CI
           ├── npm install
           ├── next build  (ignoreBuildErrors: true)
           └── deploy to CDN edge
```

Local development

```
npm run dev  →  Next.js dev server on :3000
```

---

19. Theme System

Token	Value	Usage
Page background	`#f0f4f8`	Body / main bg
Surface	`#ffffff`	Cards, panels
Navigation	`#1e3a5f`	Top nav bar
Border	`#dce5ef`	Card borders
Accent	`#1D9E75`	Buttons, highlights
Text primary	`#1e3a5f`	Headings
Text secondary	`#64748b`	Labels, metadata

---

20. Extension Points

Adding a new scenario

1. Add to `EXTRA_SCENARIOS` in `Dashboard.tsx`
2. Add its topics to `SCENARIO_AUTO_TOPICS` in `Dashboard.tsx`
3. Add a `case` in the `runScenario` switch in `client-sim.ts`

Adding a real Kafka backend

Replace the `handleTrigger` → `trigger` → `client-sim.ts` chain with an SSE stream consumer. The reducer, UI, and topic panel are unchanged.

Adding a new agent

1. Add to `AGENTS` array in `useMeshStream.ts`
2. Add a node and position to `AgentCanvas.tsx`
3. Add accent colour to `ACCENT` / `BG_LIGHT` maps
4. Emit events for the new agent in `client-sim.ts`